

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Diseño Web II

Entrega 5C

Vistas por Rol de Materias

FASE 2 — Gestión de Usuarios y Materias (Cierre)

Abril 2026

Proyecto: gestion-academica

Objetivos de Aprendizaje

Al completar esta entrega, el estudiante será capaz de:

- Crear API Routes que filtran datos según el usuario autenticado
- Implementar endpoints que usan el user.id de la sesión para filtrar (no el rol)
- Construir un componente Card reutilizable para mostrar entidades del dominio
- Crear vistas diferenciadas por rol: docente ve sus materias, estudiante ve el catálogo
- Usar CSS Grid (grid-cols) de Tailwind para layouts de cards responsivos
- Aplicar el componente Separator de shadcn/ui para dividir secciones
- Comprender la diferencia entre filtrado por sesión (RLS implícito) y filtrado explícito
- Preparar la interfaz del estudiante para la Fase 3 (inscripciones)

Prerrequisitos

Antes de comenzar, asegúrate de haber completado:

- ✓ Entregas 1A a 5B completadas y funcionando
- ✓ Al menos 5 materias creadas en el sistema
- ✓ Al menos 3 materias asignadas a un docente específico
- ✓ Al menos 1 materia sin docente asignado
- ✓ Credenciales de prueba para 3 roles: admin, docente, estudiante

i Datos de prueba: Si no tienes suficientes materias, inicia sesión como admin, ve a /admin/materias y crea las que falten. Asegúrate de asignar varias materias al mismo docente.

Teoría: Vistas por Rol y Filtrado por Sesión

Tres Perspectivas, una Misma Tabla

La tabla "materias" contiene los mismos datos, pero cada rol necesita ver información diferente:

Rol	Vista	Datos que ve	Acción disponible
Admin	/admin/materias	Todas las materias, CRUD completo	Crear, editar, eliminar (entregas 5A-5B)
Docente	/docente/materias	Solo materias asignadas a él	Ver detalle, ver estudiantes inscritos (Fase 3)
Estudiante	/estudiante/materias	Todas las materias activas	Ver catálogo, inscribirse (Fase 3)

Filtrado por Sesión vs Filtrado por Rol

Hay dos formas de filtrar datos en una API:

Tipo de filtrado	Cómo funciona	Ejemplo
Por rol	Verificamos perfil.rol y rechazamos si no es válido	Solo admins pueden acceder a /api/admin/*
Por sesión	Usamos user.id para filtrar .eq("docente_id", user.id)	El docente solo ve SUS materias

Diagrama: Filtrado por sesión del docente

```

Docente: María López (UUID: abc-123)
  ↓
GET /api/docente/materias
  ↓
SELECT * FROM materias
WHERE docente_id = 'abc-123'
  ↓
Resultado: solo las materias de María
(Cálculo I, Álgebra, Estadística)

```

✓ **Seguridad:** El filtrado por sesión es muy seguro porque el user.id viene del token JWT validado por Supabase. Un docente no puede falsificar su ID para ver materias de otro docente.

Componentes Reutilizables: MateriaCard

Tanto el docente como el estudiante verán las materias en formato Card, no en tabla. Crearemos un componente MateriaCard que acepta props y se adapta al contexto:

Patrón: Card reutilizable

```
<MateriaCard
  materia={materia}    // datos de la materia
  mostrarDocente={true} // estudiante ve el docente
  accion=<Button>      // botón personalizado por vista
/>
```

Vista Docente:

```
MAT-101
Cálculo I
6 créd | 2026-1

[Ver estudiantes]
```

Vista Estudiante:

```
MAT-101
Cálculo I
6 créd | 2026-1
Doc: M. López
[Inscribirme]
```

i **Flexibilidad:** La prop "accion" permite que cada vista pase un botón diferente. El docente verá "Ver estudiantes" (Fase 3) y el estudiante verá "Inscribirme" (Fase 3). Por ahora los crearemos como placeholders.

Paso 1 — API Route: Materias del Docente (GET)

Este endpoint devuelve únicamente las materias asignadas al docente que está logueado. Usa el `user.id` de la sesión para filtrar.

1.1 — Crear la estructura

Terminal

```
mkdir -p app/api/docente/materias
```

1.2 — Crear la API Route

Crea el archivo `app/api/docente/materias/route.ts`:

app/api/docente/materias/route.ts

```
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";

export async function GET() {
  try {
    // 1. Verificar autenticación
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    // 2. Verificar que es docente
    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "docente") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    // 3. Obtener SOLO las materias de este docente
    const { data: materias, error } = await supabase
      .from("materias")
      .select("id, codigo, nombre, creditos, semestre, created_at")
      .eq("docente_id", user.id) // Filtrado por sesión
      .order("codigo", { ascending: true });

    if (error) {
```

```

        console.error("Error al listar materias del docente:", error);
        return NextResponse.json({ error: "Error al obtener materias" }, { status: 500 });
    }

    return NextResponse.json({ materias });
} catch (err) {
    console.error("Error inesperado:", err);
    return NextResponse.json({ error: "Error interno del servidor" }, { status: 500 });
}
}

```

1.3 — Diferencias con la API admin

Aspecto	GET /api/admin/materias (5A)	GET /api/docente/materias (5C)
Rol requerido	admin	docente
Cliente Supabase	Service Role (ve todo)	Cliente normal (respeta RLS)
Filtro	Ninguno (todas)	.eq("docente_id", user.id)
Datos del docente	Incluye JOIN con docente	No necesita (ya sabe quién es)
Uso	Panel de administración	Vista personal del docente

☒ **Sin Service Role:** Este endpoint usa el cliente normal (createClient), no el Service Role. El docente solo necesita ver sus propias materias, así que no necesitamos bypassar RLS.

Paso 2 — API Route: Catálogo de Materias para Estudiante (GET)

Este endpoint devuelve todas las materias disponibles para que el estudiante las explore. Incluye los datos del docente asignado.

2.1 — Crear la estructura

Terminal

```
mkdir -p app/api/estudiante/materias
```

2.2 — Crear la API Route

Crea el archivo `app/api/estudiante/materias/route.ts`:

app/api/estudiante/materias/route.ts

```
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";

export async function GET() {
  try {
    // 1. Verificar autenticación
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    // 2. Verificar que es estudiante
    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "estudiante") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    // 3. Obtener todas las materias con datos del docente
    const { data: materias, error } = await supabase
      .from("materias")
      .select(`
        id, codigo, nombre, creditos, semestre, created_at,
        docente:usuarios_perfil!docente_id(id, nombre, apellido)
      `);
  }
}
```

```

    `)
    .order("nombre", { ascending: true });

    if (error) {
      console.error("Error al listar catálogo:", error);
      return NextResponse.json({ error: "Error al obtener materias" }, { status: 500 });
    }

    return NextResponse.json({ materias });
  } catch (err) {
    console.error("Error inesperado:", err);
    return NextResponse.json({ error: "Error interno del servidor" }, { status: 500 });
  }
}

```

2.3 — Diferencias con la API del docente

Aspecto	Docente	Estudiante
Filtro	Solo sus materias (docente_id = user.id)	Todas las materias
JOIN docente	No (ya es él)	Sí (necesita ver quién dicta)
Ordenamiento	Por código	Por nombre (más natural para buscar)
Propósito futuro	Ver inscritos (Fase 3)	Inscribirse (Fase 3)

i Misma tabla, diferente perspectiva: Ambos endpoints leen de la misma tabla "materias", pero cada uno entrega exactamente la información que el rol necesita. El estudiante necesita ver el docente asignado; el docente no.

Paso 3 — Componente Reutilizable: MateriaCard

Este componente muestra los datos de una materia en formato Card. Se usa tanto en la vista del docente como en la del estudiante, con props que controlan qué información y acciones mostrar.

3.1 — Crear el componente

Crea el archivo `components/dashboard/materia-card.tsx`:

```
components/dashboard/materia-card.tsx
import {
  Card, CardContent, CardDescription,
  CardFooter, CardHeader, CardTitle,
} from "@components/ui/card";
import { Badge } from "@components/ui/badge";
import { BookOpen, User, GraduationCap } from "lucide-react";

// Tipo para materia con docente opcional
interface MateriaCardProps {
  materia: {
    id: string;
    codigo: string;
    nombre: string;
    credits: number;
    semestre: string;
    docente?: {
      nombre: string;
      apellido: string;
    } | null;
  };
  mostrarDocente?: boolean;
  accion?: React.ReactNode;
}

export default function MateriaCard({
  materia,
  mostrarDocente = false,
  accion,
}: MateriaCardProps) {
  return (
    <Card className="flex flex-col justify-between hover:shadow-md transition-shadow">
      <CardHeader>
        <div className="flex items-center justify-between">
          <Badge variant="outline" className="font-mono">
            {materia.codigo}
          </Badge>
          <Badge variant="secondary">{materia.credits} cré&eacute;.</Badge>
        </div>
        <CardTitle className="text-lg mt-2">
          {materia.nombre}
        </CardTitle>
        <CardDescription>
          <span className="flex items-center gap-1">
            <GraduationCap className="h-3 w-3" />
            Semestre: {materia.semestre}
          </span>
        </CardDescription>
      </CardHeader>
      <CardContent>
        {accion}
      </CardContent>
    </Card>
  );
}
```

```

    </span>
  </CardDescription>
</CardHeader>

  <CardContent>
    {mostrarDocente && (
      <div className="flex items-center gap-2 text-sm text-muted-foreground">
        <User className="h-4 w-4" />
        {materia.docente
          ? `${materia.docente.nombre} ${materia.docente.apellido}`
          : "Docente no asignado"}
        </div>
      )}
    </CardContent>

    {accion && (
      <CardFooter className="pt-0">
        {accion}
      </CardFooter>
    )}
  </Card>
);
}

```

3.2 — Anatomía del componente

Prop	Tipo	Valor por defecto	Propósito
materia	objeto (requerido)	—	Datos de la materia a mostrar
mostrarDocente	boolean	false	Si true, muestra nombre del docente
accion	React.ReactNode	undefined	Botón o elemento en el footer de la card

☒ **React.ReactNode:** Este tipo acepta cualquier elemento JSX: un Button, un Link, un fragmento con múltiples botones, etc. Es la forma más flexible de inyectar contenido personalizado.

3.3 — Detalles de diseño

Analicemos las decisiones de UI en este componente:

- `hover:shadow-md transition-shadow`: la card se eleva sutilmente al pasar el mouse
- `flex flex-col justify-between`: el footer siempre queda abajo, independiente del contenido
- `font-mono` en el código: diferencia visualmente el código del nombre
- Badge con variant diferentes: `outline` para código, `secondary` para créditos

- Iconos de lucide-react: GraduationCap para semestre, User para docente

Paso 4 — Vista Docente: Mis Materias

Crearemos un Client Component que carga las materias del docente logueado y las muestra usando MateriaCard.

4.1 — Componente MisMaterias

Crea el archivo components/dashboard/docente/mis-materias.tsx:

Terminal

```
mkdir -p components/dashboard/docente
```

components/dashboard/docente/mis-materias.tsx

```
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import { Button } from "@/components/ui/button";
import { RefreshCw, Users } from "lucide-react";
import MateriaCard from "@/components/dashboard/materia-card";

interface Materia {
  id: string;
  codigo: string;
  nombre: string;
  creditos: number;
  semestre: string;
}

export default function MisMaterias() {
  const [materias, setMaterias] = useState<Materia[]>([]);
  const [loading, setLoading] = useState(true);

  const cargarMaterias = async () => {
    setLoading(true);
    try {
      const res = await fetch("/api/docente/materias");
      const data = await res.json();
      if (!res.ok) {
        toast.error(data.error || "Error al cargar materias");
        return;
      }
      setMaterias(data.materias);
    } catch {
      toast.error("Error de conexión");
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => { cargarMaterias(); }, []);
```

```

if (loading) {
  return (
    <div className="flex items-center justify-center py-12">
      <p className="text-muted-foreground">Cargando materias...</p>
    </div>
  );
}

if (materias.length === 0) {
  return (
    <div className="flex flex-col items-center justify-center py-12 gap-2">
      <p className="text-muted-foreground">
        No tienes materias asignadas aún.
      </p>
      <p className="text-sm text-muted-foreground">
        Contacta al administrador para que te asigne materias.
      </p>
    </div>
  );
}

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <p className="text-sm text-muted-foreground">
        {materias.length} materia(s) asignada(s)
      </p>
      <Button variant="outline" size="sm" onClick={cargarMaterias}>
        <RefreshCw className="mr-2 h-4 w-4" /> Refrescar
      </Button>
    </div>

    {/* Grid de cards */}
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
      {materias.map((m) => (
        <MateriaCard
          key={m.id}
          materia={m}
          mostrarDocente={false}
          accion={
            <Button variant="outline" size="sm" className="w-full disabled">
              <Users className="mr-2 h-4 w-4" />
              Ver estudiantes (próximamente)
            </Button>
          }
        </MateriaCard>
      ))}
    </div>
  </div>
);
}

```

i Botón placeholder: El botón "Ver estudiantes" está disabled por ahora. Lo activaremos en la Entrega 7A cuando creemos las vistas cruzadas (docente ve estudiantes inscritos en su materia).

4.2 — Actualizar la página del docente

Reemplaza completamente el contenido de `app/(dashboard)/docente/materias/page.tsx`:

```
app/(dashboard)/docente/materias/page.tsx
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@components/ui/card";
import { BookOpen } from "lucide-react";
import MisMaterias from "@components/dashboard/docente/mis-materias";

export default function DocenteMateriasPage() {
  return (
    <div className="flex flex-1 flex-col gap-4 p-4">
      <div className="flex items-center gap-3">
        <BookOpen className="h-8 w-8 text-primary" />
        <div>
          <h1 className="text-2xl font-bold tracking-tight">
            Mis Materias
          </h1>
          <p className="text-muted-foreground">
            Materias asignadas a tu cuenta de docente
          </p>
        </div>
      </div>

      <Card>
        <CardHeader>
          <CardTitle>Materias que dictas</CardTitle>
          <CardDescription>
            Aquí verás las materias que el administrador te ha asignado
          </CardDescription>
        </CardHeader>
        <CardContent>
          <MisMaterias />
        </CardContent>
      </Card>
    </div>
  );
}
```

☑ **Mismo patrón:** Server Component (página con Card) + Client Component (MisMaterias con fetch y estado). Este patrón se repite en todas nuestras páginas.

Paso 5 — Vista Estudiante: Catálogo de Materias

El estudiante ve todas las materias disponibles en formato card, con un botón de inscripción (placeholder por ahora) y filtro por semestre.

5.1 — Componente CatalogoMaterias

Terminal

```
mkdir -p components/dashboard/estudiante
```

Crea el archivo components/dashboard/estudiante/catalogo-materias.tsx:

components/dashboard/estudiante/catalogo-materias.tsx

```
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import { Button } from "@components/ui/button";
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from "@components/ui/select";
import { RefreshCw, ClipboardPlus } from "lucide-react";
import MateriaCard from "@components/dashboard/materia-card";

interface Docente {
  nombre: string;
  apellido: string;
}

interface Materia {
  id: string;
  codigo: string;
  nombre: string;
  creditos: number;
  semestre: string;
  docente: Docente | null;
}

export default function CatalogoMaterias() {
  const [materias, setMaterias] = useState<Materia[]>([]);
  const [loading, setLoading] = useState(true);
  const [filtroSemestre, setFiltroSemestre] = useState<string>("todos");

  const cargarMaterias = async () => {
    setLoading(true);
    try {
      const res = await fetch("/api/estudiante/materias");
      const data = await res.json();
      if (!res.ok) {
        toast.error(data.error || "Error al cargar materias");
      }
    }
  }
}
```

```

        return;
    }
    setMaterias(data.materias);
  } catch {
    toast.error("Error de conexión");
  } finally {
    setLoading(false);
  }
};

useEffect(() => { cargarMaterias(); }, []);

const semestres = [...new Set(materias.map((m) => m.semestre))].sort();
const materiasFiltradas = materias.filter(
  (m) => filtroSemestre === "todos" || m.semestre === filtroSemestre
);

if (loading) {
  return (
    <div className="flex items-center justify-center py-12">
      <p className="text-muted-foreground">Cargando catálogo...</p>
    </div>
  );
}

return (
  <div className="space-y-4">
    { /* Barra de filtros */ }
    <div className="flex items-center gap-4">
      <Select value={filtroSemestre} onValueChange={setFiltroSemestre}>
        <SelectTrigger className="w-[200px]">
          <SelectValue placeholder="Filtrar por semestre" />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="todos">Todos los semestres</SelectItem>
          {semestres.map((s) => (
            <SelectItem key={s} value={s}>{s}</SelectItem>
          ))}
        </SelectContent>
      </Select>

      <Button variant="outline" size="sm" onClick={cargarMaterias}>
        <RefreshCw className="mr-2 h-4 w-4" /> Refrescar
      </Button>

      <span className="text-sm text-muted-foreground ml-auto">
        {materiasFiltradas.length} materia(s) disponible(s)
      </span>
    </div>

    { /* Grid de cards */ }
    {materiasFiltradas.length === 0 ? (
      <div className="flex items-center justify-center py-12">
        <p className="text-muted-foreground">
          No se encontraron materias
        </p>
      </div>
    ) : (
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
        {materiasFiltradas.map((m) => (
          <MateriaCard

```



```

        key={m.id}
        materia={m}
        mostrarDocente={true}
        accion={
          <Button size="sm" className="w-full" disabled>
            <ClipboardPlus className="mr-2 h-4 w-4" />
            Inscribirse (próximamente)
          </Button>
        }
      />
    )}}
  </div>
)}
</div>
);
}

```

5.2 — Comparación: MisMaterias vs CatalogoMaterias

Aspecto	MisMaterias (Docente)	CatalogoMaterias (Estudiante)
API que consume	/api/docente/materias	/api/estudiante/materias
mostrarDocente	false	true
Filtro de semestre	No (pocas materias)	Sí (muchas materias)
Botón de acción	Ver estudiantes	Inscribirme
Estado vacío	"No tienes materias asignadas"	"No se encontraron materias"

5.3 — Actualizar la página del estudiante

Reemplaza completamente `app/(dashboard)/estudiante/materias/page.tsx`:

```
app/(dashboard)/estudiante/materias/page.tsx
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@components/ui/card";
import { BookOpen } from "lucide-react";
import CatalogoMaterias from "@components/dashboard/estudiante/catalogo-materias";

export default function EstudianteMateriasPage() {
  return (
    <div className="flex flex-1 flex-col gap-4 p-4">
      <div className="flex items-center gap-3">
        <BookOpen className="h-8 w-8 text-primary" />
        <div>
          <h1 className="text-2xl font-bold tracking-tight">
            Materias Disponibles
          </h1>
          <p className="text-muted-foreground">
            Explora las materias disponibles para inscripción
          </p>
        </div>
      </div>

      <Card>
        <CardHeader>
          <CardTitle>Catálogo de Materias</CardTitle>
          <CardDescription>
            Selecciona las materias en las que deseas inscribirte
          </CardDescription>
        </CardHeader>
        <CardContent>
          <CatalogoMaterias />
        </CardContent>
      </Card>
    </div>
  );
}
```

Resumen de Archivos

En esta entrega creaste o modificaste los siguientes archivos:

Archivo	Acción	Descripción
app/api/docente/materias/route.ts	Nuevo	GET: materias filtradas por docente logueado
app/api/estudiante/materias/route.ts	Nuevo	GET: catálogo completo con JOIN docente
components/dashboard/materia-card.tsx	Nuevo	Card reutilizable con props (materia, mostrarDocente, accion)
components/dashboard/docente/mis-materias.tsx	Nuevo	Grid de cards del docente + estado vacío
components/dashboard/estudiante/catalogo-materias.tsx	Nuevo	Grid de cards con filtro de semestre
app/(dashboard)/docente/materias/page.tsx	Modificado	Reemplaza placeholder con MisMaterias
app/(dashboard)/estudiante/materias/page.tsx	Modificado	Reemplaza placeholder con CatalogoMaterias

Verificación Final

Ejecuta `npm run dev` y prueba con 3 usuarios diferentes:

A. Vista Admin (debe seguir funcionando)

- ✓ Iniciar sesión como admin → `/admin/materias` muestra la tabla con CRUD completo
- ✓ Crear, editar y eliminar materias funciona como antes

B. Vista Docente

- ✓ Iniciar sesión como docente → navegar a `/docente/materias`
- ✓ Solo se muestran las materias asignadas a este docente
- ✓ Las materias se muestran en formato card (no tabla)
- ✓ El código aparece en Badge con font-mono
- ✓ Los créditos aparecen como Badge secondary
- ✓ El botón "Ver estudiantes" está deshabilitado
- ✓ El botón Refrescar recarga las cards
- ✓ Si el docente no tiene materias, muestra mensaje informativo

C. Vista Estudiante

- ✓ Iniciar sesión como estudiante → navegar a `/estudiante/materias`
- ✓ Se muestran TODAS las materias del sistema
- ✓ Cada card muestra el nombre del docente asignado
- ✓ Materias sin docente muestran "Docente no asignado"
- ✓ El filtro de semestre funciona correctamente
- ✓ El contador muestra el número correcto de materias filtradas
- ✓ El botón "Inscribirme" está deshabilitado

D. Grid responsivo

- ✓ En pantalla grande (lg): 3 cards por fila
- ✓ En pantalla mediana (md): 2 cards por fila

- ✓ En móvil: 1 card por fila

E. Protección de API

- ✓ Llamar a /api/docente/materias sin autenticación devuelve 401
- ✓ Llamar a /api/docente/materias como estudiante devuelve 403
- ✓ Llamar a /api/estudiante/materias como docente devuelve 403

Ejercicio Propuesto

Practica lo aprendido con estas tareas adicionales:

1. Búsqueda por texto en el catálogo: Agrega un Input de búsqueda al CatalogoMaterias que filtre por código, nombre o nombre del docente. Combina el filtro de texto con el filtro de semestre.
2. StatsCards del docente: En la vista MisMaterias, agrega encima del grid: Total de materias asignadas, Suma total de créditos de sus materias, y Semestres activos (cantidad de semestres únicos). Reutiliza el componente StatsCard de la Entrega 4A.
3. Vista de detalle: Crea una página /docente/materias/[id] que muestre los detalles completos de una materia específica. Agrega un botón "Ver detalle" en la MateriaCard del docente que navegue a esa página usando next/navigation useRouter().
4. Animación de cards: Agrega una animación sutil de entrada a las cards. Usa CSS o la librería framer-motion (npm install framer-motion). Cada card debe aparecer con un pequeño fade-in y slide-up escalonado.

i Para entregar: Implementa al menos 2 de los 4 ejercicios. Toma capturas de las 3 vistas (admin, docente, estudiante) mostrando las mismas materias desde cada perspectiva.

Próximo: Entrega 6A — Inicio de Fase 3

Con la Fase 2 completada, comenzamos la Fase 3: Inscripciones. En la Entrega 6A:

- API Route POST /api/estudiante/inscripciones para inscribirse a una materia
- API Route GET /api/estudiante/inscripciones para ver inscripciones propias
- Validaciones: no duplicar inscripción, verificar que la materia existe
- Activar el botón "Inscribirme" en el CatalogoMaterias
- Mostrar estado de inscripción (inscrito/retirado) con Badges

☑ **Preparación:** Revisa el esquema de la tabla "inscripciones" en la Entrega 1B. Los campos clave son: estudiante_id (FK), materia_id (FK), estado (CHECK: inscrito, retirado), y el UNIQUE constraint en estudiante_id + materia_id.

Fin de la Entrega 5C

Cierre definitivo de la Fase 2 — Todas las entregas 4A–5C completadas